The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

Project Checkpoint Presentation

Your Name

Your University

April 5, 2026

Outline

1. Problem Definition and Visual Intuition
2. Complexity and NP-Completeness

Overview

Goal of This Section

- Understand what the Feedback Vertex Set (FVS) problem is
- Build intuition using cycles and forests
- Visualize how removing vertices eliminates cycles

What is a Cycle?

Definition

A **cycle** is a path that starts and ends at the same vertex without repeating edges.

- Cycles represent loops in a graph
- They create dependencies or redundancies

Key Idea

If a graph has no cycles, it is called a **forest**.

What is a Forest?

Definition

A **forest** is a graph with no cycles (i.e., a collection of trees).

- Trees are connected acyclic graphs
- Forest = one or multiple disconnected trees

Observation

Our goal in FVS is to transform any graph into a forest.

Feedback Vertex Set (FVS)

Definition

Given a graph $G = (V, E)$, a **Feedback Vertex Set** is a subset $S \subseteq V$ such that removing S makes the graph acyclic.

Objective

Find the **smallest possible** such set S.

- Remove vertices $\rightarrow$ eliminate cycles
- Minimize number of removed vertices

Problem Formulation

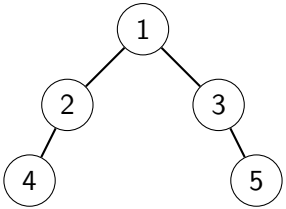
Decision Version

- Input: Graph $G = (V, E)$, integer k
- Question: Does there exist $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is acyclic?

Optimization Version

Find a minimum-size Feedback Vertex Set.

Visual Example: Tree (No Cycles)

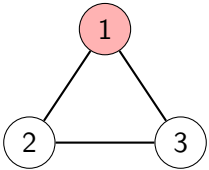


- No cycles exist
- Graph is already a forest

Result

FVS $= \emptyset$

Visual Example: Simple Cycle

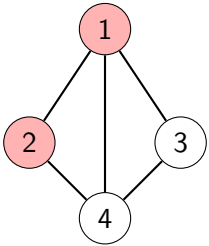


- This graph contains a cycle (triangle)
- Removing vertex 1 breaks the cycle

Result

FVS $= \{1\}$, size $= 1$

Visual Example: Multiple Cycles



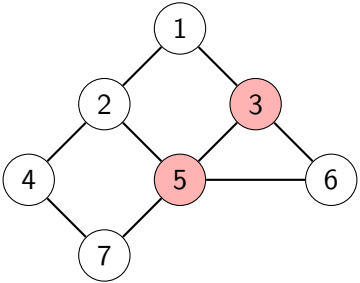
- Multiple overlapping cycles exist
- Need to remove more vertices

Result

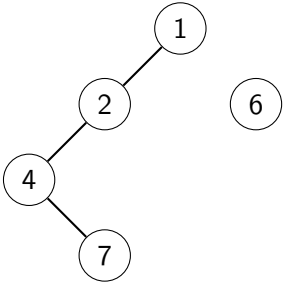
FVS $= \{1, 2\}$

Visual Example: Complex Graph → Forest

Before: Cycles



After: Forest



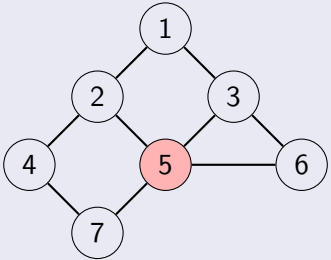
- **Before:** Multiple cycles exist, highlighted red vertices (3,5) are part of FVS.
- **After:** Removing FVS nodes breaks all cycles → Graph becomes a acyclic.

Final Verdict

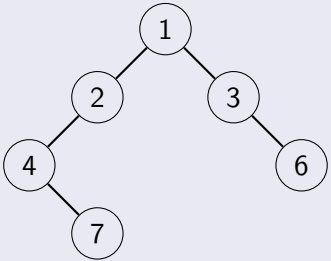
FVS $= \{3, 5\}$ → A set of vertices to remove all cycles.

Visual Example: Complex Graph → Minimal FVS → Forest

Original Graph with Cycles



Resulting Forest (Remove 5)



- Minimal Feedback Vertex Set: $\{5\}$
- Removing vertex 5 breaks all cycles → graph becomes a forest

Key Insight

Even if multiple vertices can break cycles, the **minimal FVS** is the set with the smallest size.

Real-World Motivation

- **Deadlocks in Operating Systems:** Cycles in resource allocation graph $\rightarrow$ remove critical process
- **Dependency Resolution:** Packages/modules with circular dependencies $\rightarrow$ remove one to break cycle
- **Circuit Design / Feedback Loops:** Loops in circuits $\rightarrow$ remove components to stabilize

Insight

FVS is a tool to identify minimal critical nodes whose removal resolves all cycles.

Key Observations

- Every cycle must contain at least one removed vertex
- Removing fewer vertices is always better
- Problem becomes harder as graph becomes denser

Core Insight

FVS is about selecting a minimum set of vertices that "hit" all cycles.

Complexity Overview

Goal of This Section

- Analyze the **computational complexity** of FVS
- Understand decision vs optimization versions
- Prove NP-Completeness using reductions

Decision vs Optimization

Decision Version

Given G, k: Does there exist an FVS of size $\leq k$?

Fact

Decision version is **NP-Complete**

Optimization Version

Find the minimum-size FVS

Fact

Optimization version is **NP-Hard**

Main Claim

Theorem

The decision version of Feedback Vertex Set is **NP-Complete**.

Proof Strategy

1. Show FVS $\in$ NP
2. Reduce Vertex Cover $\leq_p$ FVS

Step 1: FVS is in NP

Key Idea (NP Definition)

A problem is in NP if a solution can be **verified in polynomial time**.

- Given a set S
- Remove S from graph
- Check if graph has cycles (DFS)

Time Complexity

$O(V + E)$

Conclusion

FVS $\in$ NP (verification is polynomial)

Step 2: Reduction (VC $\rightarrow$ FVS)

Known Fact

Vertex Cover is NP-Complete

Goal

Transform VC instance into FVS instance

- Each edge becomes a cycle
- FVS must break all cycles

Step 2: Reduction (VC $\rightarrow$ FVS)

Vertex Cover (VC) Problem

Find smallest $S \subseteq V$ such that every edge has at least one endpoint in S.

Construction: Given VC instance (G, k)

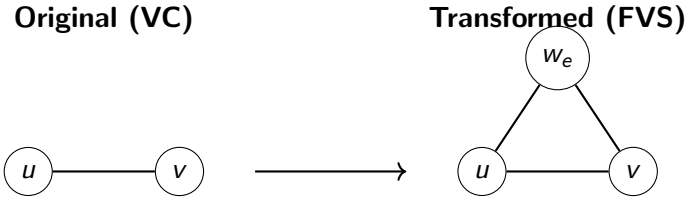
1. For each edge $e = (u, v) \in E(G)$:
   - Add new vertex w_e
   - Form triangle: edges (u, v), (u, w_e), (v, w_e)
2. Keep same parameter $k' = k$

Theorem

G has VC of size $\leq k \Leftrightarrow G'$ has FVS of size $\leq k$

Time: $O(|E|)$ (polynomial)

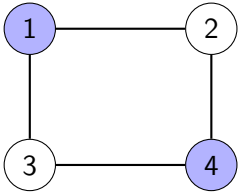
Reduction Construction (Graph)



- Each edge becomes a triangle
- Triangle = cycle → must be broken

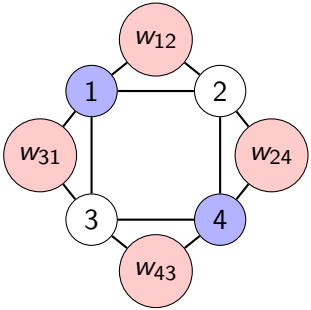
VC → FVS: Full Graph Illustration

Original Graph



VC = $\{1, 4\}$

Transformed Graph



Each triangle = one edge

Key Insight

Breaking all triangles = covering all edges

Correctness of Reduction

Claim

G has a VC of size $\leq k \iff G'$ has an FVS of size $\leq k$

Forward:

- VC selects endpoints $\rightarrow$ every triangle broken

Backward:

- FVS must hit each triangle
- Implies every edge is covered

Polynomial Time Reduction

- Add one vertex per edge
- Linear size increase

Conclusion

Reduction runs in polynomial time

Reverse Reduction (FVS $\rightarrow$ VC)

Reverse Direction

Shows Vertex Cover and FVS are polynomially equivalent

Construction: Given FVS instance (G, k)

1. Replace each vertex $v \in V$ with triangle $\{v_1, v_2, v_3\}$
2. For each edge $(u, v) \in E$, connect triangles with cross edges
3. Set $k' = 2|V| + k$

Theorem

G has FVS of size $\leq k \Leftrightarrow G'$ has VC of size $\leq 2|V| + k$

Key Insight:

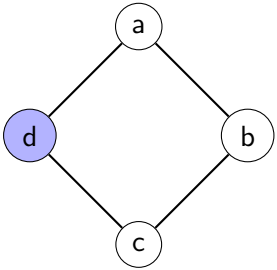
- Each triangle needs ≥ 2 vertices in VC
- Triangles with 3 vertices in VC, $\leftrightarrow$ vertices in FVS

FVS→ VC: Visual Example

Idea

Show structural similarity between problems

Original Graph



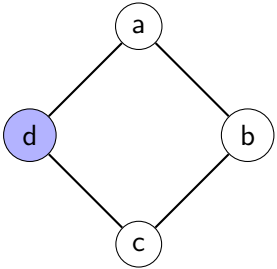
Construction

- Each vertex → triangle
- Selecting all 3, ↔ remove vertex
- Selecting 2, ↔ keep vertex

FVS $= \{d\}$

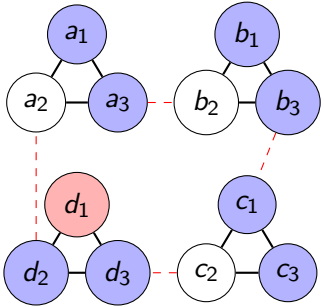
FVS→ VC: Visual Example

Original Graph G:



FVS: $\{d\}$ (size=1)
Removing d breaks the 4-cycle

Transformed Graph G':



Dashed = original edges , Each triangle = one vertex in G

Vertex Cover in G':(with d1) is a FVS in G'

Final Conclusion

We Have Proven

- FVS $\in$ NP
- Vertex Cover $\leq_p$ FVS
- VC is NP-Complete

Final Verdict

Feedback Vertex Set (Decision) is NP-Complete

Implication

- Optimization version is NP-Hard
- No known polynomial-time exact algorithm

Algorithm Selection Criteria

Essential Criteria

1. **Implementability**
   - Feasible in 6-week timeframe
   - Well-documented in literature
   - Reasonable debugging complexity
2. **Research Potential**
   - Novel comparisons possible
   - Room for improvements
   - Publishable results
3. **Complementary Nature**
   - Different problem sizes
   - Different paradigms
   - Interesting comparisons

Additional Criteria

4. **Practical Relevance**
   - Real-world applications
   - Industry interest
   - Addresses practical needs
5. **Educational Value**
   - Learn important techniques
   - Transferable skills
   - Deep algorithmic insights

Goal

Select algorithms that maximize research impact while ensuring feasibility

Selected Algorithms for Implementation

Three-Algorithm Approach (Maximum Research Impact)

We will implement three complementary algorithms covering the entire solution spectrum:

1. **Iterative Compression** (Exact FPT Algorithm)
   - Guarantees optimal solution for small-medium k
   - Time: $O(5^k \cdot kn^2)$ — FPT breakthrough (2004)
   - Research: Hybrid approaches, parallelization, adaptive branching
2. **Kernelization + Bounded Search Tree** (Enhanced Exact)
   - Preprocessing with reduction rules + exact search
   - Kernel reduction: 50-90% for many graph types
   - Research: New reduction rules, smart branching strategies
3. **Memetic Algorithm** (Advanced GA + Local Search)
   - Handles large instances (n $>$ 1000)
   - Combines population evolution with local optimization
   - Research: Problem-specific operators, ML integration, hybrid exact-metaheuristic

Algorithm 1: Iterative Compression (IC)

Core Concept

Given FVS of size $k+1$, compress it to size k through smart enumeration

Theoretical Significance:

- Most important FPT technique
- Published in STOC/FOCS
- Demonstrates advanced understanding

Implementation:

- Complexity: Medium
- Expected range: $k \leq 30$
- Validation on small instances

Research Directions:

1. **Hybrid Kernelization + IC**
   - Apply reduction rules before IC
   - Potential 2-5x speedup
2. **Adaptive Branching**
   - Learn which vertices to branch first
   - Use graph structure metrics
3. **Parallelization**
   - Independent branches in parallel
   - Multi-core speedup potential
4. **Improved Compression**
   - Smart partition enumeration
   - Structure-based pruning

Iterative Compression for FVS

Idea: Build solution incrementally and compress when too large.

Algorithm

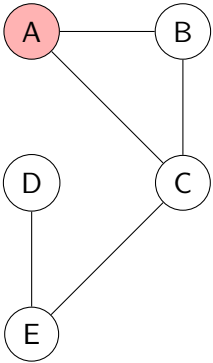
```
Order vertices v_1, ..., v_n
F ← {v_1}
for i = 2 to n do
    F ← F ∪ {v_i}
    if |F| = k + 1 then
        for all partitions (F_1, F_2) of F do
            find Y ⊆ V \ F_2
            if F_1 ∪ Y is an FVS then
                update F and break
            end if
        end for
        if no update performed then
            return NO
        end if
    end if
end for
return F if |F| ≤ k
```

Key Insight: Compress size $k+1$ to k

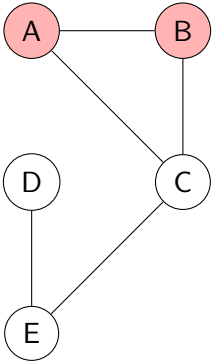
Time: $O(5^k \cdot n^2)$

Simulation

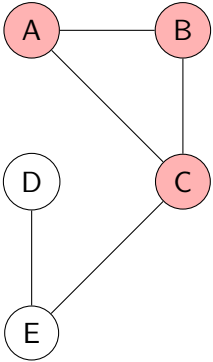
Simulation of FVS (k=2) on 5 vertices
Step 1: $F = \{A\}$



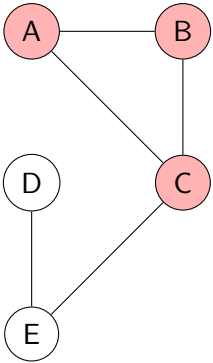
Step 2: $F = \{A, B\}$



Step 3: $F = \{A, B, C\}$ (overflow)

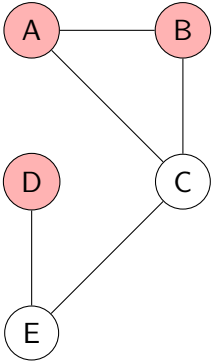


Step 3: $F = \{A, B, C\}$ (overflow)

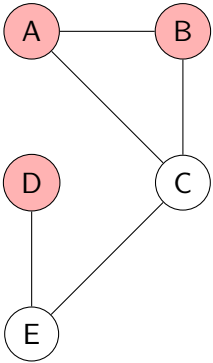


Fix: Remove C $\Rightarrow F = \{A, B\}$

Step 4: Add D (Overflow)

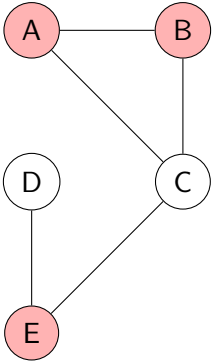


Step 4: Add D (Overflow)

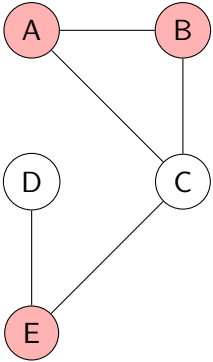


Fix: Remove D $\Rightarrow F = \{A, B\}$

Step 5: Add E (Overflow)

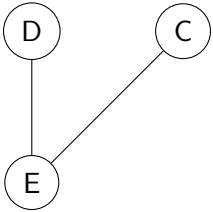


Step 5: Add E (Overflow)



Fix: Remove E $\Rightarrow F = \{A, B\}$

Final Answer



$F = \{A, B\}$

Algorithm 2: Kernelization + Bounded Search Tree

Core Concept

Reduce graph size via preprocessing rules, then apply exact search on smaller kernel

Reduction Rules:

- Degree 0, 1, 2 vertices
- Self-loops, duplicate edges
- Crown decomposition
- Flower rule

Search Strategy:

- Smart vertex selection
- Bounded depth: k
- Branching factor: ≤ 4

Research Directions:

1. **New Reduction Rules**
   - Graph-specific rules
   - Theoretical contribution
2. **Rule Application Order**
   - Optimize ordering
   - ML-based prediction
3. **Kernelization Bounds**
   - Tighter bounds for special classes
   - Experimental validation
4. **Smart Branching**
   - Centrality-based selection
   - ML-guided search

Kernelization + Bounded Search Tree

Idea: Reduce the graph, then branch on remaining cycles.

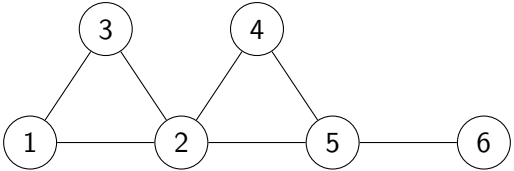
Algorithm

```
Apply reduction rules:
    remove degree ≤ 1 vertices
    contract degree-2 vertices
    include self-loop vertices
Obtain kernel (G', k')
function Search(G', k')
if G' is acyclic then
    return solution
end if
if k' < 0 then
    return NO
end if
find a cycle C
for each v ∈ C do
    recurse on (G' − v, k' − 1)
end for
```

Key Insight: Small kernel $\Rightarrow$ small search tree

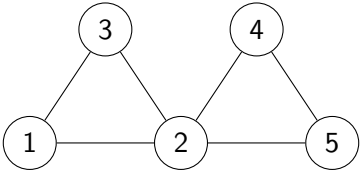
Time: $O(4^k + n^2)$

Step 1: Original Graph

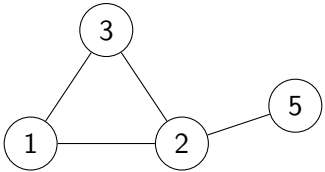


Step 2: Remove degree ≤ 1 vertices

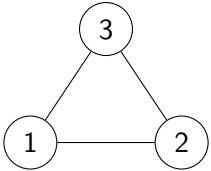
Vertex 6 removed



Step 3: Contract degree-2 vertex (4)

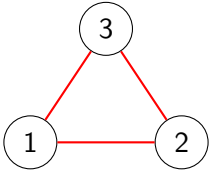


Step 4: Remove degree ≤ 1 vertex (5)

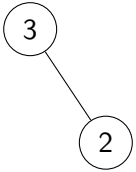


Step 5: Kernel Graph (G')

Remaining cycle

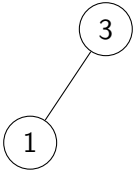


Step 6: Branch - Remove vertex 1



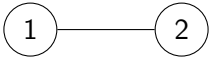
Graph becomes acyclic

Step 7: Branch - Remove vertex 2



Graph becomes acyclic

Step 8: Branch - Remove vertex 3



Graph becomes acyclic

Algorithm 3: Memetic Algorithm (GA + Local Search)

Core Concept

Population evolution + local optimization for large-scale instances

Components:

- Smart initialization (greedy + random)
- FVS-aware crossover operators
- Centrality-based mutation
- Problem-specific local search

Practical Relevance:

- Industry standard
- Handles $n > 1000$ vertices
- Near-optimal (5-10% gap)

Research Directions:

1. **Cycle-Preserving Crossover**
   - Preserve cycle-breaking structures
   - Novel contribution
2. **Hybrid Exact-Metaheuristic**
   - Use IC for small components
   - GA for large core
3. **ML Integration**
   - Predict promising vertices
   - Learn operator selection
4. **Multi-Objective**
   - Minimize size + maximize robustness
   - Pareto frontier

Memetic Algorithm for FVS

Idea: Evolutionary search combined with local optimization.

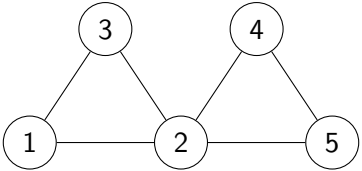
Algorithm

```
Encode solution as a permutation of vertices
Decode by removing vertices until graph is acyclic
Initialize population (greedy + random)
for each generation do
    Evaluate fitness: −|FVS|
    Select parents (tournament)
    Crossover (cycle-aware)
    Mutation (swap cycle-heavy vertices)
    Local search (hill-climbing)
end for
return best solution found
```

Key Insight: Exploration + refinement

Type: Heuristic (near-optimal)

Step 1: Original Graph

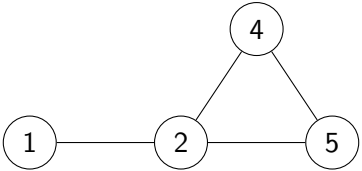


Step 2: Encode Solution

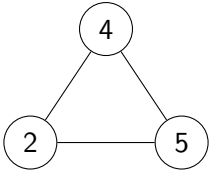
Example permutation:

$$[3, 1, 5, 2, 4]$$

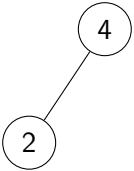
Step 3: Remove vertex 3



Step 4: Remove vertex 1



Step 5: Remove vertex 5



Graph is now acyclic $\rightarrow$ Stop decoding

Step 6: Fitness Evaluation

$$\text{Removed vertices} = \{3,1,5\}$$

$$\text{Fitness} = -|FVS| = -3$$

Step 7: Selection (Tournament)

Select best solutions from population

Example:

$$[3, 1, 5, 2, 4], \quad [2, 4, 1, 5, 3]$$

Step 8: Crossover

Parent 1: [3,1,5,2,4]
Parent 2: [2,4,1,5,3]

Child:

[3,4,1,2,5]

Step 9: Mutation

Before:

$$[3, 4, 1, 2, 5]$$

After swap:

$$[3,1,4,2,5]$$

Step 10: Local Search

Try improving solution:

Remove fewer vertices if possible

Example improvement:

$$\text{FVS size: } 3 \rightarrow 2$$

Step 11: Best Solution

Best FVS found:

$$\{3,2\}$$

Graph becomes acyclic